

ROS2 Day2 과제 보고서

학번: 2026406076

이름: 정창규

제출일: 2026-09-10

목차

1. Task01 – Custom Message를 이용한 Vector Topic 통신

- 1.1 과제 개요
- 1.2 Custom Message 및 Publisher, Subscriber 구현
- 1.3 Topic 통신 및 실행 결과
- 1.4 느낀 점

2. Task02 – Qt GUI를 이용한 turtlesim 제어

- 2.1 과제 개요
- 2.2 Qt GUI 및 turtlesim 제어 프로그램 구현
- 2.3 WASD 조작 및 도형 그리기
- 2.4 실행 결과
- 2.5 느낀 점

3. Task03 – Qt GUI를 이용한 Talker / Listener 통신

- 3.1 과제 개요
- 3.2 Talker / Listener 프로그램 구현
- 3.3 실행 준비 및 작동 영상
- 3.4 느낀 점

1. Task01 – Custom Message를 이용한 Vector Topic 통신

1.1 과제 개요

Task01에서는 ROS2에서 Custom Message를 직접 생성하고, C++로 Publisher와 Subscriber를 작성하여 Topic 통신을 구현하였다. 통신에 사용한 Custom Message는 정수형 데이터와 정수형 배열을 함께 전달할 수 있도록 구성하였다. Publisher에서는 터미널에서 여러 개의 정수를 입력받아 vector에 저장한 뒤 메시지로 발행하도록 하였고, Subscriber에서는 전달받은 데이터의 개수와 vector에 저장된 값을 터미널에 출력하도록 하였다. 또한 Custom Message가 정상적으로 생성되었는지 `ros2 interface show` 명령어를 이용하여 확인하고, 실제 Publisher와 Subscriber를 실행하여 입력한 데이터가 정상적으로 전달되는지 확인하였다.

1.2 Custom Message 및 Publisher, Subscriber 구현

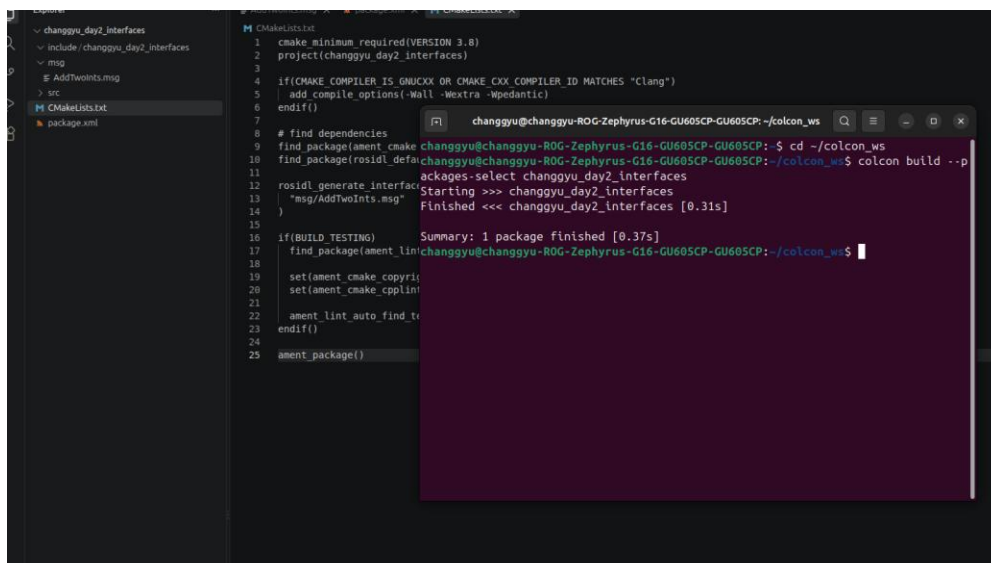
Vector 형태의 데이터를 Topic으로 전달하기 위해 `changgyu_day2_interfaces` 패키지를 생성하고 Custom Message를 작성하였다. Custom Message 파일은 `AddTwoInts.msg`로 작성하였으며, 내용은 다음과 같이 구성하였다.

```
int64 a
int32[] b
```

a에는 Publisher에서 입력받은 vector의 원소 개수를 저장하고, b에는 실제로 입력받은 정수 데이터를 저장하도록 하였다. Custom Message 패키지가 정상적으로 생성되는지 확인하기 위해 다음 명령어를 사용하여 빌드하였다.

```
cd ~/colcon_ws
colcon build --packages-select changgyu_day2_interfaces
```

빌드 결과 Summary: 1 package finished가 출력되어 Custom Message 패키지가 정상적으로 빌드된 것을 확인하였다.

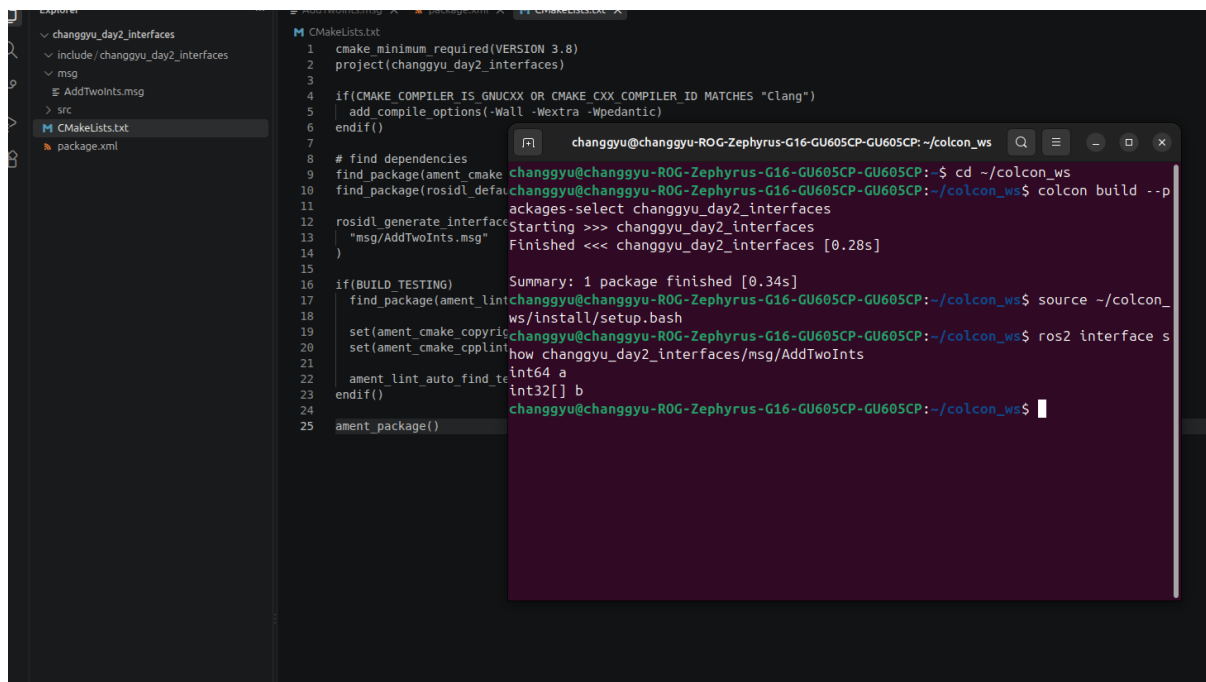


그다음 작성한 Custom Message가 ROS2에 정상적으로 등록되었는지 확인하기 위해 다음 명령어를 사용하였다.

```
source ~/colcon_ws/install/setup.bash
ros2 interface show changgyu_day2_interfaces/msg/AddTwoInts
```

실행 결과 다음과 같이 작성한 메시지 구조가 출력되었다.

```
int64 a
int32[] b
```



Publisher와 Subscriber는 changgyu_day2_task1_cpp 패키지에 작성하였으며, 헤더 파일과 소스 파일을 각각 분리하여 구성하였다.

Publisher에서는 터미널에서 여러 개의 정수를 입력받아 vector에 저장하도록 하였다. 이후 입력된 데이터의 개수는 a에 저장하고, 실제 vector 데이터는 b에 저장하여 /day2/vector_data Topic으로 발행하도록 구현하였다.

```
msg.a = data.size();
msg.b = data;
```

Subscriber에서는 동일한 /day2/vector_data Topic을 구독하고, 메시지가 수신되면 callback 함수에서 전달받은 데이터의 개수와 vector 내부의 값을 터미널에 출력하도록 작성하였다.

1.3 Topic 통신 및 실행 결과

Publisher와 Subscriber가 정상적으로 통신하는지 확인하기 위해 각각 별도의 터미널에서 노드를 실행하였다.

Publisher는 다음 명령어를 이용하여 실행하였다.

```
cd ~/colcon_ws
source install/setup.bash
ros2 run changgyu_day2_task1_cpp publisher_node
```

Subscriber는 별도의 터미널에서 다음 명령어로 실행하였다.

```
cd ~/colcon_ws
source install/setup.bash
ros2 run changgyu_day2_task1_cpp subscriber_node
```

Publisher 실행 후 여러 개의 정수를 입력하여 vector 형태로 데이터를 발행하였다.

```
10 20 30 40 50
20 40 60 80 100
1 2 3 4 5 6 7
```

첫 번째와 두 번째 입력에서는 5개의 값이 입력되어 Published 5 values가 출력되었고, 마지막 입력에서는 7개의 값이 입력되어 Published 7 values가 출력되었다.

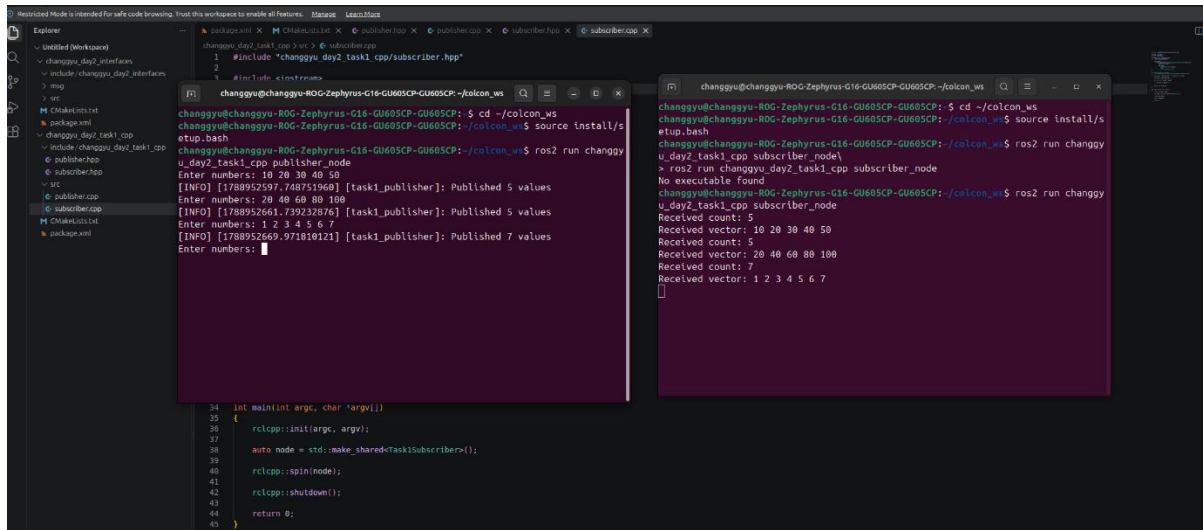
Subscriber에서는 Publisher에서 발행한 데이터의 개수와 vector 값이 다음과 같이 정상적으로 출력되는 것을 확인하였다.

```
Received count: 5
Received vector: 10 20 30 40 50

Received count: 5
Received vector: 20 40 60 80 100

Received count: 7
Received vector: 1 2 3 4 5 6 7
```

이를 통해 입력하는 vector의 크기가 달라져도 Custom Message를 이용하여 데이터의 개수와 실제 값을 정상적으로 전달할 수 있음을 확인하였다.



1.4 느낀 점

이번 과제를 통해 ROS2에서 기본 메시지만 사용하는 것이 아니라 필요한 데이터 형식에 맞게 Custom Message를 직접 만들어 사용할 수 있다는 것을 알 수 있었다.

처음에는 int64 값과 int32[] 배열을 하나의 메시지 안에서 어떻게 사용하는지 헷갈렸지만, Publisher에서 vector의 크기와 실제 데이터를 각각 저장하고 Subscriber에서 이를 다시 출력하는 과정을 통해 Custom Message의 구조를 이해할 수 있었다.

또한 입력한 vector의 길이가 달라져도 Subscriber에서 정상적으로 데이터가 전달되는 것을 확인 하면서 배열 형태의 데이터를 Topic 통신에 사용하는 방법을 익힐 수 있었다.

2. Task02 – Qt GUI를 이용한 turtlesim 제어

2.1 과제 개요

Task02에서는 Qt GUI와 ROS2를 이용하여 turtlesim의 거북이를 제어하는 프로그램을 구현하였다.

Qt Designer를 이용하여 W, A, S, D 버튼과 Triangle, Square, Circle, Reset 버튼을 배치하였으며, 버튼을 클릭하거나 키보드를 입력하여 거북이를 이동하거나 회전할 수 있도록 하였다.

또한 Day1 Task03에서 구현했던 삼각형, 사각형, 원 그리기 기능을 GUI에서도 사용할 수 있도록 하였으며, 도형마다 선의 색상과 굵기가 다르게 적용되도록 구성하였다.

GUI에는 현재 선택된 Mode와 linear.x, angular.z 값을 표시하여 거북이에게 전달되는 속도 값을 확인할 수 있도록 하였다.

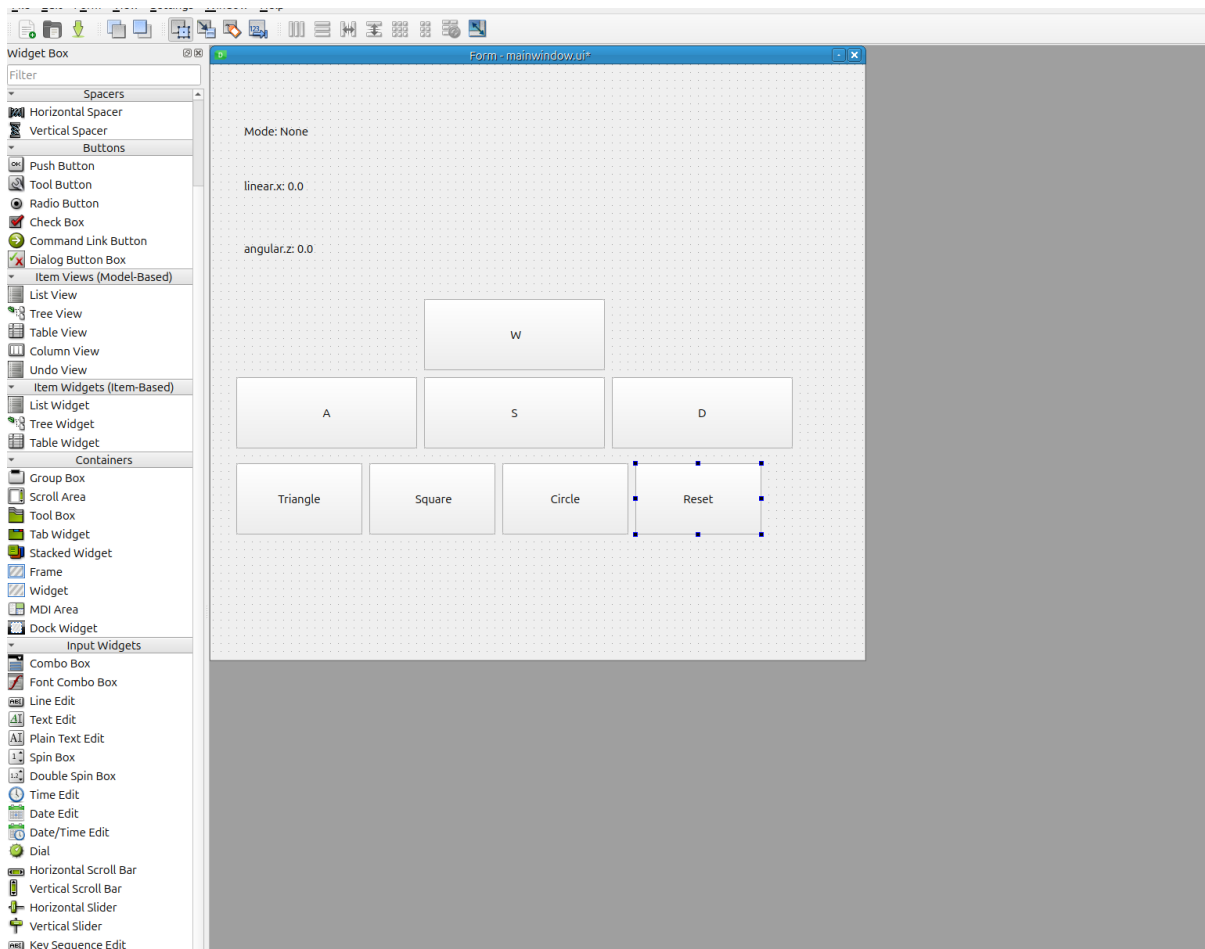
2.2 Qt GUI 및 turtlesim 제어 프로그램 구현

Qt Designer를 이용하여 turtlesim을 제어하기 위한 GUI를 구성하였다.

GUI에는 현재 선택된 도형 모드를 확인할 수 있는 Mode 표시 영역과 거북이의 선속도와 각속도를 확인할 수 있는 linear.x, angular.z 표시 영역을 배치하였다.

거북이의 이동을 제어하기 위해 W, A, S, D 버튼을 배치하였으며, 각각 전진, 좌회전, 후진, 우회전 기능을 수행하도록 하였다.

또한 Triangle, Square, Circle 버튼을 이용하여 그릴 도형을 선택할 수 있도록 하였으며, Reset 버튼을 추가하여 turtlesim을 초기 상태로 되돌릴 수 있도록 하였다.



GUI는 .ui 파일로 작성하였으며, C++ 프로그램에서 해당 UI를 불러와 각 버튼과 동작을 연결하였다.

예를 들어 W 버튼을 누르면 다음과 같이 전진 동작을 실행하도록 연결하였다.

```
connect(  
    ui->wButton,  
    &QPushButton::clicked,  
    this,  
    [this]()  
    {  
        moveForward();  
    });
```

키보드에서도 동일한 동작을 수행할 수 있도록 keyPressedEvent()에서 W, A, S, D 입력을 구분하였다.

```

case Qt::Key_W:
    moveForward();
    break;

case Qt::Key_S:
    moveBackward();
    break;

case Qt::Key_A:
    turnLeft();
    break;

case Qt::Key_D:
    turnRight();
    break;

```

거북이의 실제 이동은 /turtle1/cmd_vel Topic에 geometry_msgs/msg/Twist 메시지를 발행하는 방식으로 구현하였다.

linear.x에는 직선 이동 속도를 설정하고, angular.z에는 회전 속도를 설정하여 전진, 후진, 좌회전, 우회전을 제어하였다.

```

msg.linear.x = linear;
msg.angular.z = angular;

pub_->publish(msg);

```

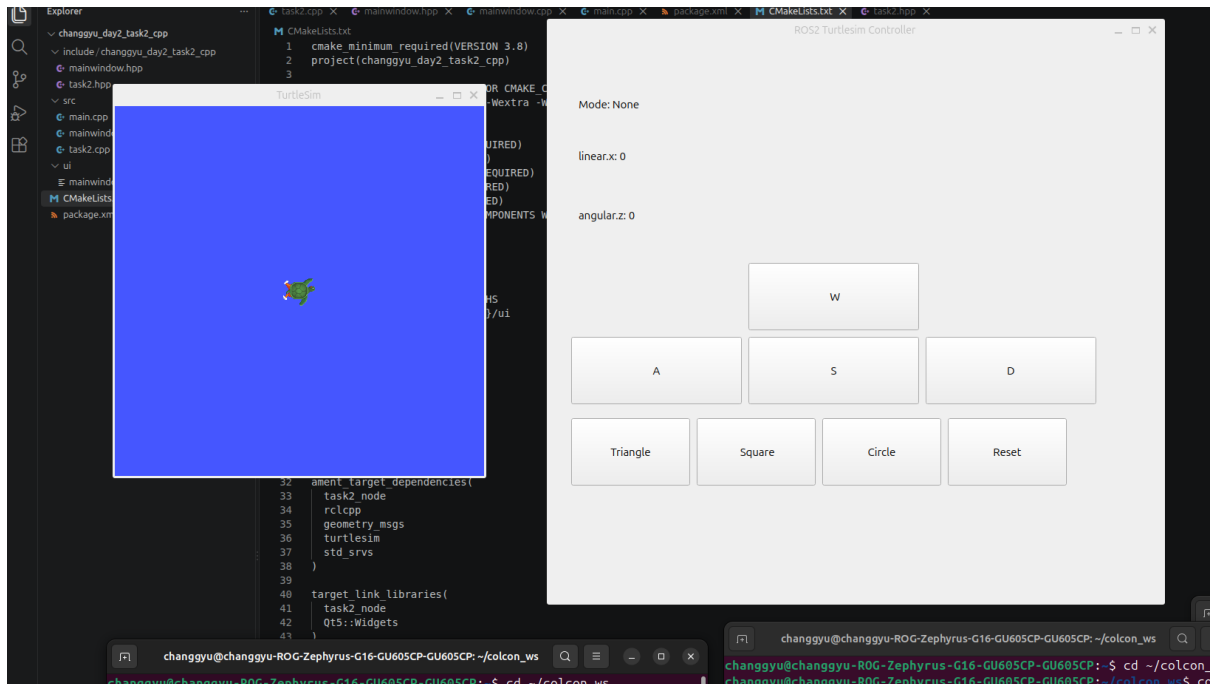
또한 GUI에서 거북이의 현재 이동 값을 확인할 수 있도록 linear.x와 angular.z 값을 QLabel에 표시하였다.

```

ui->linearLabel->setText(
    QString("linear.x: %1").arg(linear));

ui->angularLabel->setText(
    QString("angular.z: %1").arg(angular));

```

2.3 WASD 조작 및 도형 그리기

W, A, S, D 키와 GUI 버튼을 이용하여 turtlesim의 거북이를 이동할 수 있도록 구현하였다.

W는 전진, S는 후진, A는 좌회전, D는 우회전 동작을 수행하도록 하였다. 삼각형과 사각형은 전진과 회전을 반복하여 그릴 수 있도록 하였으며, 선택된 도형 모드에 따라 회전 시간을 다르게 적용하였다.

삼각형 모드에서는 약 120도씩 회전하도록 설정하였다.

```
if (mode_ == 1)
{
    task2_->move(
        0.0,
        1.57,
        1.33);
}
```

사각형 모드에서는 약 90도씩 회전하도록 설정하였다.

```
else
{
    task2_->move(
        0.0,
        1.57,
        1.0);
}
```

원 모드에서는 직선 이동과 회전을 따로 수행하지 않고 linear.x와 angular.z 값을 동시에 적용하여 한 바퀴 회전하도록 구현하였다.

```
if (mode_ == 3)
{
    task2_->move(1.0, 1.0, 6.28);
}
```

도형을 쉽게 구분할 수 있도록 turtlesim의 set_pen 서비스를 이용하여 각 도형마다 선의 색상과 굵기도 다르게 설정하였다.

```
// Triangle
task2_->setPen(255, 0, 0, 2);

// Square
task2_->setPen(0, 255, 0, 6);

// Circle
task2_->setPen(0, 0, 255, 10);
```

삼각형은 빨간색에 굵기 2, 사각형은 초록색에 굵기 6, 원은 파란색에 굵기 10으로 설정하였다.

W, A, S, D 키와 GUI 버튼을 이용한 거북이의 이동 과정과 Triangle, Square, Circle 모드에 따른 도형 그리기, 선의 색상 및 굵기 변화는 작동 영상으로 별도 첨부하였다.

2.4 실행 결과

turtlesim 노드와 Task02 패키지를 각각 실행한 뒤 Qt GUI를 이용하여 거북이의 이동과 도형 그리기 기능을 확인하였다.

GUI의 W, A, S, D 버튼과 키보드 입력을 이용하여 전진, 후진, 좌회전, 우회전이 정상적으로 동작하였으며, 이동할 때마다 linear.x와 angular.z 값이 GUI에 표시되는 것을 확인하였다.

Triangle, Square, Circle 버튼을 이용하여 도형 모드를 변경할 수 있었고, 선택한 모드에 따라 회전 방식과 선의 색상 및 굵기가 다르게 적용되었다. 또한 Reset 버튼을 이용하여 turtlesim을 초기 상태로 되돌릴 수 있었다.

삼각형, 사각형, 원을 그리는 실제 동작 과정과 GUI 조작 과정은 작동 영상으로 별도 첨부하였다.

2.5 느낀 점

이번 과제를 진행하면서 ROS2와 Qt GUI를 함께 사용하여 거북이를 제어하는 방법을 익힐 수 있었다.

기존에는 터미널이나 키보드 입력만 이용하여 turtlesim을 조작했지만, 이번에는 Qt Designer를 이용하여 직접 버튼과 상태 표시 화면을 구성하고 이를 C++ 코드와 연결하였다. 이를 통해 GUI

에서 발생한 버튼 입력이 ROS2의 Topic 통신과 연결되어 실제 거북이의 움직임으로 이어지는 과정을 이해할 수 있었다.

또한 linear.x와 angular.z 값을 GUI에 표시하면서 거북이의 이동과 회전에 어떤 값이 사용되는지 확인할 수 있었고, set_pen 서비스를 이용하여 도형마다 선의 색상과 굵기를 다르게 적용할 수 있었다.

3. Task03 – Qt GUI를 이용한 Talker / Listener 통신

3.1 과제 개요

Task03에서는 Qt GUI와 ROS2 Topic 통신을 이용하여 문자열 데이터를 주고받는 Talker와 Listener 프로그램을 구현하였다.

Talker에서는 Qt GUI의 입력창에 문자열을 입력한 뒤 Publish 버튼을 누르면 해당 문자열이 Topic으로 발행되도록 하였다. Listener에서는 같은 Topic을 Subscribe하고, 문자열을 수신하면 GUI의 Label에 수신한 내용이 표시되도록 구현하였다.

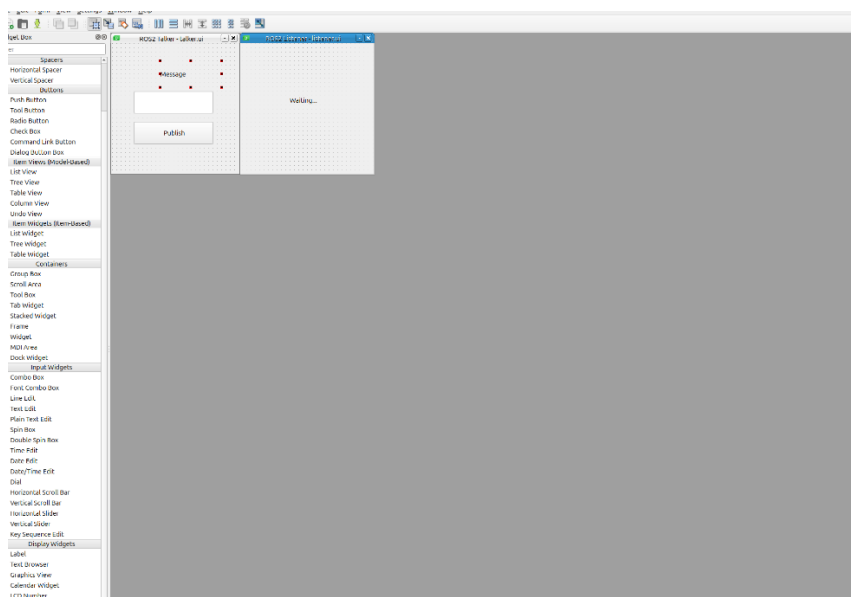
Talker와 Listener는 각각 별도의 Qt GUI로 구성하였으며, Qt Designer를 이용하여 .ui 파일을 작성하였다. 이를 통해 Qt에서 입력한 데이터가 ROS2 Topic을 통해 다른 노드로 전달되고, 수신한 데이터가 GUI에 다시 표시되는 과정을 확인하였다.

3.2 Talker / Listener 프로그램 구현

Talker와 Listener는 각각 Qt Designer를 이용하여 별도의 GUI로 구성하였다.

Talker GUI에는 문자열을 입력하기 위한 QLineEdit과 메시지를 발행하기 위한 Publish 버튼을 배치하였다. 입력창의 objectName은 messageEdit, 버튼은 publishButton으로 설정하였다.

Listener GUI에는 수신한 문자열을 표시하기 위한 QLabel을 배치하였으며, 초기 상태에서는 Waiting...이 표시되도록 하였다.



Talker에서는 Publish 버튼을 누르면 입력창에 작성한 문자열을 가져와 /day2/chat Topic으로 발행하도록 구현하였다.

```
msg.data =  
    ui->messageEdit->text().toStdString();  
  
publisher_>publish(msg);
```

Listener에서는 동일한 /day2/chat Topic을 Subscribe하도록 설정하였다. 메시지가 수신되면 callback 함수가 실행되고, 전달받은 문자열을 GUI의 messageLabel에 표시하도록 구현하였다.

```
ui->messageLabel->setText(  
    QString::fromStdString(msg->data));
```

따라서 Talker에서 문자열을 입력한 뒤 Publish 버튼을 누르면 Listener의 Waiting... 문구가 수신한 문자열로 변경되도록 하였다.

3.3 실행 준비 및 작동 영상

Talker와 Listener의 Topic 통신을 확인하기 위해 각각 별도의 터미널에서 노드를 실행하였다.

먼저 Listener 노드를 다음 명령어로 실행하였다.

```
cd ~/colcon_ws  
source install/setup.bash  
ros2 run changgyu_day2_task3_cpp listener_node
```

이후 다른 터미널에서 Talker 노드를 실행하였다.

```
cd ~/colcon_ws  
source install/setup.bash  
ros2 run changgyu_day2_task3_cpp talker_node
```

프로그램이 실행되면 Talker GUI의 입력창에 문자열을 입력하고 Publish 버튼을 눌러 메시지를 발행하였다.

Listener에서는 동일한 /day2/chat Topic을 Subscribe하고 있기 때문에 메시지를 수신하면 Waiting...으로 표시되어 있던 Label이 Talker에서 입력한 문자열로 변경되도록 하였다.

Talker에서 문자열을 입력하여 Publish하고, Listener에서 해당 문자열이 수신되어 Label 값이 변경되는 실제 동작 과정은 작동 영상으로 별도 첨부하였다.

3.4 느낀 점

이번 과제를 통해 Qt GUI와 ROS2의 Topic 통신을 함께 사용하는 방법을 익힐 수 있었다.

이전에는 Publisher와 Subscriber의 통신 결과를 터미널에서만 확인했지만, 이번에는 Qt GUI를 이

용하여 문자열을 직접 입력하고 Publish 버튼을 눌러 메시지를 전송하도록 구현하였다. 또한 Listener에서 수신한 문자열을 Label에 표시하면서 ROS2에서 전달받은 데이터를 GUI에 연결하는 과정을 확인할 수 있었다.

Talker에서 입력한 문자열이 /day2/chat Topic을 통해 Listener로 전달되고, 수신한 값에 따라 Waiting...이 실제 문자열로 변경되는 것을 보면서 Publisher와 Subscriber의 동작 흐름을 더 쉽게 이해할 수 있었다.